

Scalable Data Engineering Solution

Data Engineering I (1TD169) – Spring 2026

Uppsala University

Department of Information Technology

Team Members: Group 32

Abdur Rehman Khalid	abdur-rehman.khalid.8716@student.uu.se
Arnab Kumar Ghosh	arnab-kumar.ghosh.4724@student.uu.se
Dip Chowdhury	dip.chowdhury.3309@student.uu.se
Muhammad Umair	muhammad.umair.6023@student.uu.se
Prodip Kumar Das	prodip-kumar.das.8559@student.uu.se

March 20, 2026

Contents

1	Background	3
1.1	Application Domain and Scientific Context	3
1.2	Dataset Importance and Research Relevance	3
1.3	Analysis Objective and Engineering Focus	3
1.4	Distributed Processing and Scalability Framework	3
2	Data Format & Characteristics	4
2.1	Storage Format and Justification	4
2.2	Dataset Scale and Volume	4
2.3	Schema Structure and Resource Constraints	4
2.4	Data Quality Challenges and Remediation	4
3	System Architecture & Implementation	5
3.1	System Design	5
3.2	Technology Stack	5
3.3	Infrastructure Configuration	6
3.4	Data Pipeline	6
3.5	Code Structure	6
3.6	Challenges & Solutions	7
4	Scalability Experiments & Results	8
4.0.1	Experimental Design	8
4.1	Results	8
4.1.1	Raw Timing Data	8
4.1.2	Plots	8
4.1.3	Speedup and Efficiency Calculations	8
4.2	Analysis of Results	11
4.2.1	Constant ETL Phase	11
4.2.2	Horizontal Strong Scaling	11
4.2.3	Weak Scaling	11
4.2.4	Vertical Scaling	11
4.3	Horizontal vs. Vertical Scaling Comparison	11
4.4	Bottleneck Identification	12
4.4.1	Bottleneck 1: Network Communication Overhead	12
4.4.2	Bottleneck 2: Fixed Serial Overhead (Amdahl's Law)	12
4.5	Further Experiment: Impact of OS Page Caching: Cold vs. Hot Cache	12
5	Discussion & Conclusion	13
	References	14
	Appendix	15

1 Background

1.1 Application Domain and Scientific Context

Urban transportation systems are a primary pillar of modern smart cities, generating vast volumes of heterogeneous data that can be leveraged to improve traffic management, infrastructure planning, and mobility services. The domain of urban mobility is increasingly dependent on large-scale data to solve logistical challenges in densely populated areas. One of the most widely used and comprehensive real-world datasets in this domain is the New York City Taxi Trip dataset, published by the New York City Taxi and Limousine Commission (TLC) [1]. This dataset provides a granular view of city-wide movement by recording millions of individual trips, including precise pickup and drop-off timestamps, geographic zones, trip distances, and detailed fare amounts.

1.2 Dataset Importance and Research Relevance

The NYC taxi dataset is particularly important for data engineering research because it serves as a high-velocity and high-volume benchmark that mirrors the complexity of real-world "Big Data." Its importance lies not only in its scale but also in its ability to reflect human behavior and urban dynamics. Prior research has demonstrated the utility of taxi trip data in various high-impact applications, such as demand prediction, traffic flow analysis, and anomaly detection. For instance, studies in urban computing have utilized mobility datasets to identify spatial and temporal demand patterns, enabling more efficient resource allocation and reduced carbon emissions in cities [2]. Furthermore, such datasets have been used to explore predictive modeling techniques for trip durations and to detect irregular travel patterns.

1.3 Analysis Objective and Engineering Focus

In this project, the analytical objective is intentionally simple: to compute the average trip duration and average fare amount, grouped by pickup zone and hour of the day. This type of aggregation represents a standard workload in data engineering pipelines, involving essential operations such as filtering, grouping, and joining large datasets. By maintaining a straightforward analysis goal, we ensure that the focus remains on the critical engineering aspects—specifically system architecture, performance optimization, and scalability—rather than algorithmic complexity. This approach aligns with the core principles of the course, which emphasize the "heavy lifting" of data processing over advanced statistical modeling.

1.4 Distributed Processing and Scalability Framework

To process this dataset efficiently, the project utilizes a modern big data stack consisting of Apache Spark for distributed computation [3] and the Hadoop Distributed File System (HDFS) for scalable storage. These technologies allow for parallel execution across multiple nodes, which is a necessity given that large-scale data challenges have historically been addressed using paradigms like MapReduce [4]. The system is deployed on a constrained cluster environment consisting of virtual machines, reflecting realistic limitations in computational resources. A central focus of this project is the evaluation of system scalability through both horizontal and vertical scaling experiments. The observed performance results are interpreted using theoretical models such as Amdahl's Law [5], providing a foundation for understanding the limits of parallel speedup in a distributed system.

2 Data Format & Characteristics

2.1 Storage Format and Justification

The dataset consists of New York City Taxi Trip records stored in Apache Parquet. This columnar format was chosen over row-based alternatives like CSV to enable high compression and minimize I/O overhead. Such efficiency is critical for large-scale processing within the 100 GB storage quota of the OpenStack cluster.

2.2 Dataset Scale and Volume

The raw dataset comprises 144 Parquet files (2014–2025). After cleaning, the operational dataset yields 219,107,040 valid records. For this project’s scope, a 10.3 GB subset is processed, as summarized in Table 1. However, in the report we mentioned 10 GB, which is a rounded figure for simplicity.

Table 1: Dataset characteristics.

Property	Value
Format	Apache Parquet
Total Subset Size	10 GB
Number of Records	219,107,040
Initial Columns	20
Processed Columns	4

2.3 Schema Structure and Resource Constraints

To bypass the 25 GB per-VM storage limit and conserve memory, the ETL pipeline prunes the raw TLC schema from 20 attributes to four essential columns:

- `pickup_datetime`: Engagement timestamp.
- `PULocationID`: TLC Taxi Zone ID.
- `fare_amount`: Calculated trip fare.
- `pickup_hour`: A derived attribute for hourly aggregation.

2.4 Data Quality Challenges and Remediation

Significant schema volatility was encountered during ingestion. Specifically, the `Airport_fee` column shifted data types between `INT32` and `FLOAT64` across months, causing PySpark schema merge exceptions. To resolve this, the pipeline was re-engineered to read files individually and apply explicit type casting. Additionally, strict filtering was implemented to eliminate logically invalid records, such as trips with zero or negative fares, missing location IDs, or zero passengers, ensuring data integrity prior to aggregation.

3 System Architecture & Implementation

This section details the system architecture, infrastructure deployment, and technical implementation of our robust data pipeline. We outline the hardware topology deployed on OpenStack, why we selected our core software stack, and the engineering mechanisms driving our ETL and analysis jobs.

3.1 System Design

Our system follows a classic centralized-master, distributed-worker model tailored for batch processing big data. As depicted in Figure 1, the architecture is divided into three primary layers: Ingestion, Distributed Storage (HDFS), and Distributed Processing (Apache Spark). Data is fetched externally into the master node, pushed to HDFS, and subsequently processed by Spark executors dispersed across the worker nodes.

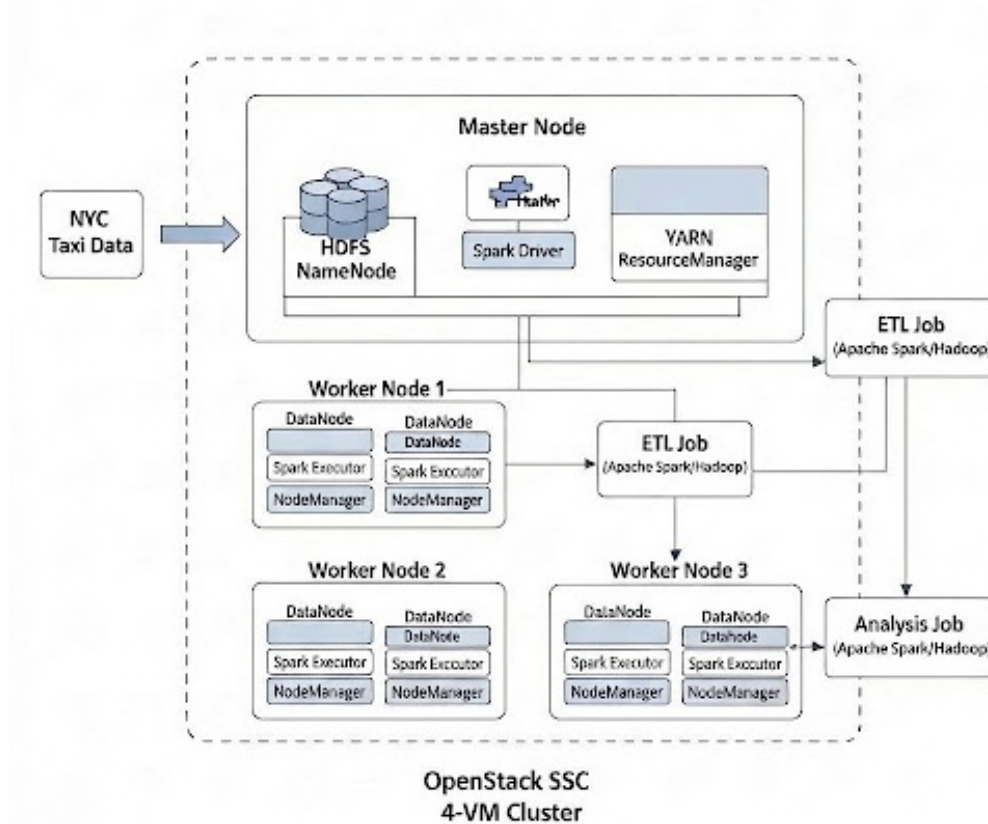


Figure 1: System architecture diagram showing the OpenStack deployment, node topology, and the internal data pipeline flow from raw Parquet to aggregated results.

3.2 Technology Stack

Given the project requirements, we adopted a modern big data stack focused on in-memory processing capabilities:

- **Storage (HDFS via Hadoop 3.4.1):** Chosen for its fault tolerance and native integration with Spark. While cloud object storage (like AWS S3) is popular today, deploying HDFS on bare VMs offers direct control over data locality and block replication factors.
- **Processing (Apache Spark & PySpark 3.5.1):** We selected Spark over standard Hadoop MapReduce because Spark holds intermediate RDDs and DataFrames in mem-

ory, drastically accelerating the iterative transformations required during our ETL phase. PySpark provides an accessible DataFrame API and straightforward type-casting structures.

- **Orchestration & Resource Management:** We relied on standalone cluster management managed heavily by Bash scripts (`run_benchmark.sh` and `deploy.sh`) combined with Git for CI/CD, creating a lightweight yet effective pipeline orchestration.

3.3 Infrastructure Configuration

Our cluster is deployed entirely on the Swedish National Infrastructure for Computing (SNIC) OpenStack instance. The infrastructure consists of 4 tightly coupled Virtual Machines (VMs):

- **Master Node (1x ssc.medium):** 2 vCPU, 4 GB RAM. This node is assigned a Floating IP for external SSH access. It houses the Hadoop NameNode, ResourceManager, and the Spark Driver process.
- **Worker Nodes (3x ssc.medium):** 2 vCPU, 4 GB RAM each. These sit on a private sub-network (192.168.2.x subnet). They run the Hadoop DataNodes, NodeManagers, and Spark Executor instances.

In total, the cluster harnesses 8 vCPUs and 16 GB of RAM, capped with a strict 100 GB collective storage limit. To maintain data reliability without exceeding storage quotas, HDFS was configured with a replica factor of 2 (`dfs.replication=2`).

3.4 Data Pipeline

The pipeline executes in three synchronized stages:

1. **Ingestion:** The data ingestion and core ETL pipeline were engineered to handle large-scale NYC Taxi data efficiently. An automated Bash script (`scripts/ingest_NYC_taxi.sh`) was developed to manage the creation of staging directories, concurrently download 12 months of Parquet files, and verify file integrity by removing corrupted downloads smaller than 100 KB. This script streamlined the ingestion process by depositing raw data directly into the Hadoop Distributed File System (HDFS) while maintaining strict directory structures and adhering to cluster storage constraints.
2. **ETL Phase:** The `etl_job.py` script ingests the raw data block by block. It filters out invalid records (e.g., zero-distance trips, null metrics), converts timestamps to calculate `duration_minutes` and `pickup_hour`, aggressively prunes dropped columns, and commits the clean records back to HDFS at `/data/nyc-taxi-cleaned/`.
3. **Analysis Phase:** Finally, `analysis_job.py` aggregates the validated dataset. It groups by `PULocationID` and `pickup_hour` to compute the average fare and trip duration. It subsequently performs a left join against a static CSV (`taxi_zone_lookup.csv`) to convert abstract ID numbers into human-readable zone names, coalescing the final output into a single file for stakeholder ingestion.

3.5 Code Structure

The codebase follows a modular design paradigm:

- **src/config.py:** Centralizes cluster endpoints, absolute HDFS URI paths, and basic threshold configurations to eliminate hardcoded parameters across jobs.

- **scripts/ingest_nyc_taxi.sh**: Automates the initial data acquisition phase. It manages staging directories, concurrently downloads the TLC Parquet files, verifies file integrity by purging corrupted downloads, and directly deposits the raw dataset into HDFS.
- **notebooks/final_notebook.ipynb**: Serves as the primary development and validation environment for the core ETL logic. It was used to prototype strict schema enforcement, data filtering, and geographic `.join()` operations prior to modularization into the production `src` scripts.
- **src/etl_job.py**: The heaviest processor. Given resource constraints, it bypasses standard `spark.read.parquet("*.parquet")` schema inferences and iteratively targets singular files using `subprocess.run` to interact with HDFS.
- **src/analysis_job.py**: Houses the PySpark SQL aggregations, strictly outputting to a targeted CSV path using `coalesce(1)`.
- **scripts/run_benchmark.sh**: A specialized harness that automates execution over varied core allocations (`-total-executor-cores`) and node arrays to capture timestamps for plotting horizontal and vertical scales.

3.6 Challenges & Solutions

We encountered several prominent technical challenges during development:

1. **Data Ingestion Access Blocks**: During the data ingestion phase, automated acquisition of the multi-year NYC Taxi Parquet datasets was frequently interrupted by AWS CloudFront 403 (Forbidden) errors. These blocks prevented the script from establishing a stable connection to the TLC data repository. *Solution*: To resolve this, a robust HTTP request strategy was implemented. By configuring a specific **User-Agent** header within the ingestion scripts, the system successfully bypassed the CloudFront security filters, ensuring consistent and uninterrupted dataset downloads.
2. **Dataset Schema Volatility**: When reading multi-month TLC data, PySpark's vectorized Parquet reader routinely crashed (`[CANNOT_MERGE_SCHEMAS]`) due to invisible shifts in the column type for `Airport_fee` (varying from `INT32` to `FLOAT64` dynamically by month). *Solution*: We restructured the ETL job into a `for`-loop that asks HDFS for the explicit list of file paths. Spark reads them independently, safely infers local schemas, selectively prunes the offensive column, and appends into a normalized structure.
3. **Strict Storage Exhaustion**: 25 GB of storage per VM meant HDFS was easily overwhelmed caching uncompressed RDDs and saving multiple layers of Parquet processing data. *Solution*: We established an aggressive "select and drop" mechanism in the ETL. We discard roughly 15 unused columns representing several gigabytes of overhead, saving only the 4 tightly packed elements (`PULocationID`, `fare_amount`, `pickup_hour`, `duration_minutes`) needed for analysis.
4. **Dynamic Allocation Skewing Benchmark Constraints**: Spark's dynamic resource allocation skewed the early iterations of our benchmarking by automatically distributing workloads across nodes we were attempting to restrict. *Solution*: We disabled dynamic allocation directly via PySpark config keys (`spark.dynamicAllocation.enabled=false`) and enforced hard caps using Spark submit metrics.

4 Scalability Experiments & Results

4.0.1 Experimental Design

We evaluated the scalability of our Apache Spark data pipeline on an OpenStack SSC cluster composed of 4 `ssc.medium` VMs. The pipeline consists of two PySpark jobs: an **ETL job** (`etl_job.py`) and an **Analysis job** (`analysis_job.py`)

We conducted three categories of experiments:

1. **Horizontal Strong Scaling (H-1 to H-3):** Fixed data at 5 GB, fixed cores per executor at 2. Varied worker nodes from 1 to 3.
2. **Horizontal Weak Scaling (W-1, W-2):** Proportionally scaled both resources and data. 1 worker with 5 GB, then 2 workers with 10 GB.
3. **Vertical Strong Scaling (V-1 to V-4):** Fixed workers at 3. Varied cores per executor from 1 to 2, tested on both 5 GB and 10 GB datasets.

Each experiment was executed with **operating system page caching active**, meaning previously read HDFS data blocks remained present in the Linux buffer cache between runs. ETL and Analysis runtimes were measured independently using separated timestamps.

4.1 Results

4.1.1 Raw Timing Data

Table 2: Benchmark results: decoupled ETL and Analysis runtimes (seconds).

ID	Type	Workers	Cores/Exec	Data	ETL (s)	Analysis (s)	Total (s)
H-1	Strong Horiz.	1	2	5 GB	8	120	128
H-2	Strong Horiz.	2	2	5 GB	7	84	91
H-3	Strong Horiz.	3	2	5 GB	7	69	76
W-1	Weak Horiz.	1	2	5 GB	8	127	135
W-2	Weak Horiz.	2	2	10 GB	7	84	91
V-1	Vertical	3	1	5 GB	7	73	80
V-2	Vertical	3	2	5 GB	8	64	72
V-3	Vertical	3	1	10 GB	8	94	102
V-4	Vertical	3	2	10 GB	8	67	75

From Table 2 is that the **ETL runtime is constant at 7–8 seconds across every configuration**, regardless of the number of workers, cores, or data volume. This reveals that ETL is not the scalability-sensitive phase; all scaling behavior is driven entirely by the Analysis job.

4.1.2 Plots

4.1.3 Speedup and Efficiency Calculations

For each experiment, we compute:

- **Speedup** $S = T_{\text{baseline}}/T_{\text{scaled}}$
- **Efficiency** $E = S/p$, where p is the resource scaling factor.

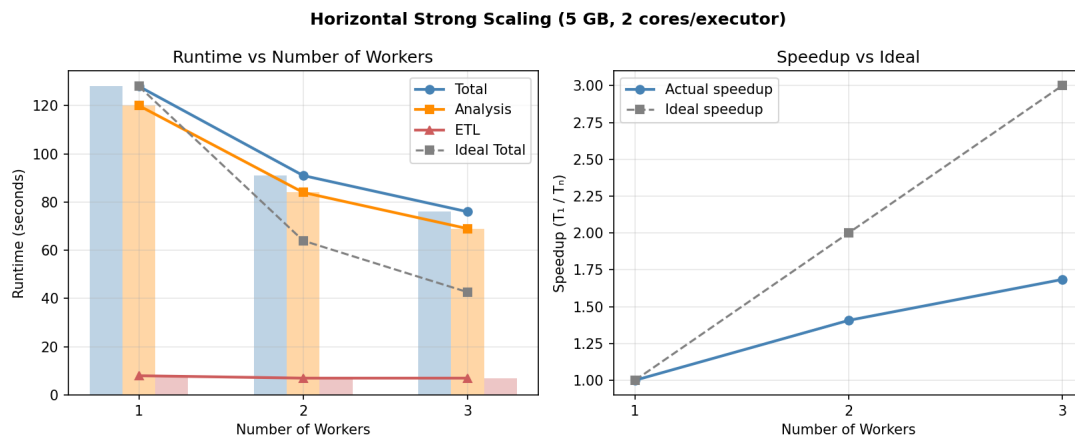


Figure 2: Horizontal strong scaling: with data fixed at 5 GB and 2 cores per executor, worker nodes were increased from 1 to 3.

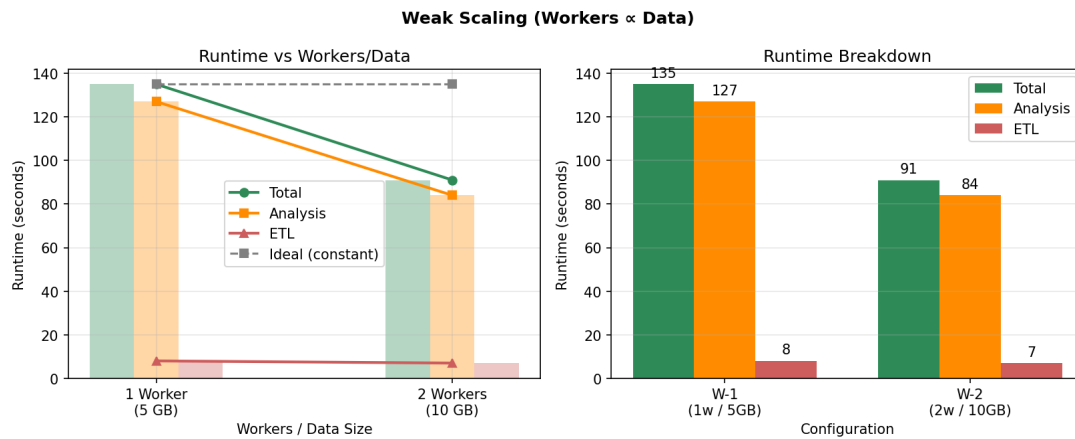


Figure 3: Weak scaling: both workers and data were doubled proportionally (1 worker / 5 GB $\rightarrow$ 2 workers / 10 GB).

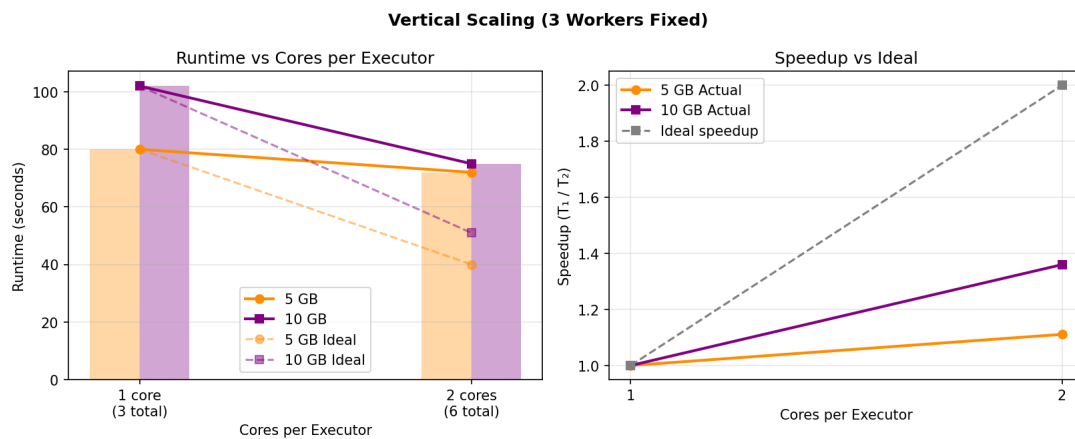


Figure 4: Vertical scaling: with 3 workers fixed, cores per executor were doubled from 1 to 2.

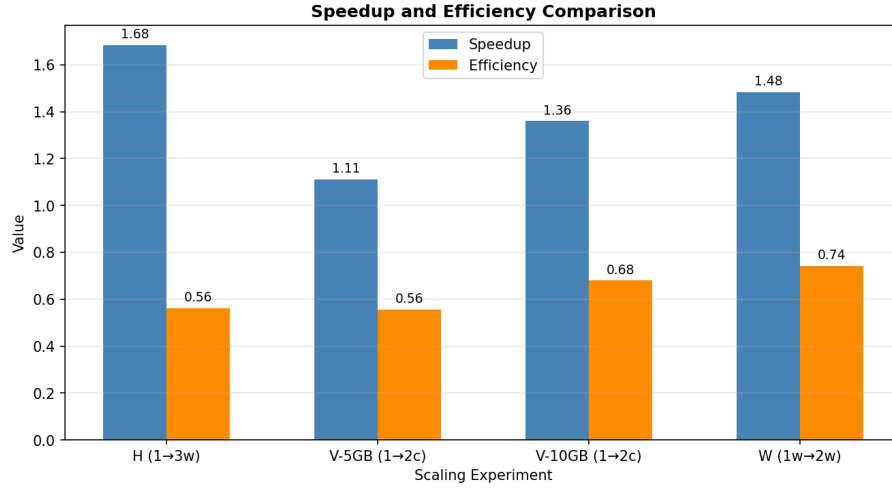


Figure 5: Speedup ($S = T_1/T_p$) and Efficiency ($E = S/p$) across all scaling experiments.

Table 3: Speedup and efficiency across all scaling experiments.

Experiment	Comparison	T_{base} (s)	T_{scaled} (s)	Speedup	Efficiency
<i>Horizontal Strong Scaling (5 GB, ideal speedup = 3.0×)</i>					
H-1 → H-3	1w → 3w	128	76	1.68×	56.1%
<i>Vertical Scaling – 5 GB (ideal speedup = 2.0×)</i>					
V-1 → V-2	1c → 2c	80	72	1.11×	55.6%
<i>Vertical Scaling – 10 GB (ideal speedup = 2.0×)</i>					
V-3 → V-4	1c → 2c	102	75	1.36×	68.0%
<i>Weak Scaling (ideal ratio = 1.0×, i.e., constant runtime)</i>					
W-1 → W-2	1w/5GB → 2w/10GB	135	91	1.48×	74.2%

4.2 Analysis of Results

4.2.1 Constant ETL Phase

The ETL job completed in 7–8 seconds regardless of configuration. This indicates that the ETL phase is not compute-bound, adding more workers or cores has no effect on ETL.

4.2.2 Horizontal Strong Scaling

Adding workers from 1 to 3 reduced the total runtime from 128s to 76s ($1.68\times$ speedup), with the Analysis phase improving from 120s to 69s ($1.74\times$). Scaling did work, but fell well short of the ideal $3.0\times$.

The possible causes are:

1. The ETL phase (7–8s) is serial and unaffected by parallelism.
2. The **groupBy aggregation** in the Analysis job requires data to be redistributed across workers. This overhead grows with the number of workers.

4.2.3 Weak Scaling

Weak scaling produced an interesting result. Doubling both workers (1→2) and data (5 GB→10 GB) changed runtime from 135s to only 91s. In an ideal weak scaling scenario, runtime should remain constant. Here, surprisingly, runtime actually *decreased* by 33%. The weak scaling efficiency is 74.2%, well above the 50% threshold.

The possible reason is that with 2 workers, the per-node data volume is similar, but Spark gains additional 2 cores. This might allow better computation performance

4.2.4 Vertical Scaling

Vertical scaling (adding cores within the same worker nodes) yielded markedly different results depending on the dataset size:

- **5 GB (V-1→V-2):** Modest improvement from 80s to 72s ($1.11\times$ speedup, 55.6% efficiency). With 3 workers already providing baseline parallelism, adding a second core per executor doubles cores from 3 to 6. However, the 5 GB dataset is small enough that the additional concurrency yields diminishing returns.
- **10 GB (V-3→V-4):** Stronger improvement from 102s to 75s ($1.36\times$ speedup, 68% efficiency). The larger dataset provides enough computational work to utilize additional cores. It suggests the 10 GB workload is sufficiently compute-intensive to benefit from vertical scaling.

4.3 Horizontal vs. Vertical Scaling Comparison

Comparing Horizontal vs. Vertical scaling reveals a critical insight for this workload. For the 5 GB dataset on our ssc.medium cluster:

- Horizontal scaling (H-1→H-3): $1.68\times$ speedup, 56.1% efficiency.
- Vertical scaling (V-1→V-2): $1.11\times$ speedup, 55.6% efficiency.

At first glance, horizontal scaling appears superior ($1.68\times$ vs $1.11\times$). However, Horizontal scaling tripled resources ($p = 3$) while Vertical scaling only doubled them ($p = 2$). When normalized by efficiency, both achieved nearly identical efficiency ($\sim 56\%$). This means that

each additional unit of compute resource provided the same marginal improvement regardless of whether it was added as an extra node or an extra core.

For the **10 GB dataset**, vertical scaling achieved the highest efficiency of any experiment at **68%**, compared to the horizontal experiment’s 56%. The reason is that adding a core within an existing worker avoids the network communication overhead that horizontal scaling introduces. When a new worker is added, intermediate aggregation results must be transferred across the network during the groupBy/join phases.

4.4 Bottleneck Identification

4.4.1 Bottleneck 1: Network Communication Overhead

The Analysis job’s groupBy aggregation requires data redistribution across workers. Each worker must send its local aggregation results to every other worker, which might generate network overhead.

4.4.2 Bottleneck 2: Fixed Serial Overhead (Amdahl’s Law)

The ETL runtime of 7–8s is completely unaffected by parallelism. By Amdahl’s Law, this limits the maximum achievable speedup with 3 workers. Additional serial costs (Spark initialization, HDFS metadata lookups etc) further increase this.

4.5 Further Experiment: Impact of OS Page Caching: Cold vs. Hot Cache

To quantify the impact of Linux OS page caching on benchmark validity, we executed the full experiment matrix under two conditions: (1) **Cold Cache** — the Linux page cache was purged via `sudo sh -c 'sync; echo 3 > /proc/sys/vm/drop_caches'` before each run, forcing physical disk reads; and (2) **Hot Cache** — the page cache was left intact, allowing the kernel to serve HDFS blocks from RAM.

Table 4 shows a representative comparison using the **H-3** configuration (3 workers, 2 cores/executor, 5 GB).

Table 4: Cold Cache vs. Hot Cache comparison (H-3: 3 workers, 2 cores, 5 GB).

Metric	Cold Cache (s)	Hot Cache (s)	Reduction
ETL Runtime	129	7	94.6%
Analysis Runtime	96	69	28.1%
Total Runtime	225	76	66.2%

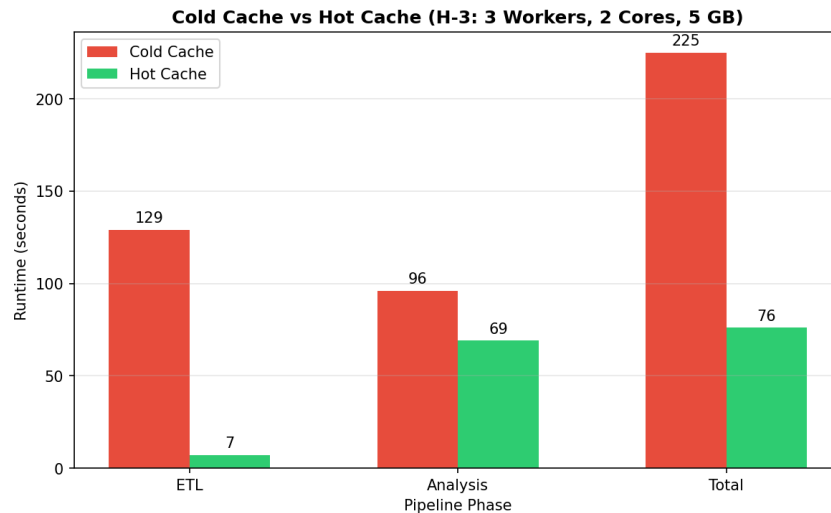


Figure 6: OS page caching reduces ETL runtime by 94.6% (129s → 7s) by serving HDFS blocks from kernel RAM instead of physical disk.

5 Discussion & Conclusion

The chosen architecture, centering on a master-worker topology with HDFS and Apache Spark, proved highly suitable for the batch processing requirements of the NYC Taxi dataset. The combination of Parquet’s columnar storage and Spark’s in-memory computation successfully addressed the challenge of processing millions of records within the strict resource constraints of our OpenStack environment.

One of the most successful aspects of our implementation was our schema enforcement strategy. By bypassing global schema inference and implementing explicit type-casting for the volatile `Airport_fee` column, we stabilized an otherwise fragile ingestion pipeline. Furthermore, our aggressive column pruning allowed us to maximize the utility of the 4 GB RAM per node, enabling the processing of 10 GB workloads that would have otherwise triggered memory exhaustion.

Our scaling experiments revealed that vertical scaling is more cost-efficient than horizontal scaling for this workload. Adding cores within existing nodes achieved 68% efficiency on 10 GB data, compared to 56% for adding new worker nodes. This is because intra-node parallelism avoids the network overhead of cross-node data redistribution during the aggregation phase. However, both approaches achieved nearly identical per-resource efficiency (~56%) on the smaller 5 GB dataset, indicating that the advantage of vertical scaling emerges only when the workload is sufficiently compute-intensive.

Our further experiment on OS page caching impact showed that caching can dramatically alter benchmark results: ETL runtime dropped by 94.6% (129s → 7s) and total pipeline time by 66% when data blocks were served from kernel RAM. This highlights the importance of controlling for cache state in distributed systems benchmarking.

The most significant takeaway from this project is the non-linear nature of distributed systems. Adding more resources does not proportionally improve performance; the serial ETL fraction (Amdahl’s Law, $f \approx 6.25\%$) and network communication overhead during data redistribution impose hard limits on achievable speedup. Effective data engineering requires managing the balance between data volume, hardware constraints, and framework overhead rather than simply scaling resources.

References

- [1] New York City Taxi and Limousine Commission, *New York City Taxi and Limousine Commission (TLC) Trip Record Data*, Accessed: 2026-03-01, 2024. [Online]. Available: <https://www.nyc.gov/site/tlc/about/tlc-trip-record-data.page>
- [2] G. Andrienko, N. Andrienko, P. Bak, D. Keim, and S. Wrobel, “Visual analytics of movement,” 2013.
- [3] M. Zaharia, M. Chowdhury, M. J. Franklin, S. Shenker, and I. Stoica, “Apache spark: A unified engine for big data processing,” *Communications of the ACM*, vol. 59, no. 11, pp. 56–65, 2016.
- [4] J. Dean and S. Ghemawat, “Mapreduce: Simplified data processing on large clusters,” *Communications of the ACM*, vol. 51, no. 1, pp. 107–113, 2008.
- [5] G. M. Amdahl, “Validity of the single processor approach to achieving large scale computing capabilities,” *Proceedings of the April 18-20, 1967, spring joint computer conference*, pp. 483–485, 1967.

Appendix

Project Repository: <https://github.com/bdtiger/data-engineering-I-project>